FACULTÉ DES SCIENCES
Unité de formation et de recherche
université angers

L2 MI - 2025/2026
UE Développement Web
David Lesaint

CC JS : Session 1 - 1h - Sur machine

Téléchargez l'archive **cc-21-js-sujet.tgz** de l'espace Moodle. Après extraction, renommez le répertoire obtenu **nom-prenom** où **nom** et **prenom** sont vos nom et prénom écrits en minuscules et sans espace (par ex. **delafontaine-jean**). A l'issue de l'examen, archivez ce répertoire (`tar czf nom-prenom.tgz nom-prenom`) et déposez l'archive sur Moodle.
Les exercices sont indépendants et peuvent être traités dans l'ordre de votre choix. Pour visualiser ce qui est attendu, consultez ce démonstrateur.

Exercice 1. (DOM et Évènements)
Le fichier **page.html** contient divers éléments (paragraphes, titres, formulaire, ...) et importe le fichier **page.js**. Compléter ce dernier sans modifier le premier pour répondre aux questions.
1. L'option `Maine-et-Loire` du menu déroulant doit être sélectionnée par défaut au chargement de la page.
2. Les paragraphes de classe `info` apparaissent en italique et de couleur grise au chargement de la page.
3. Tous les labels du document, dont le texte contient un point d'interrogation, sont mis en bleu au chargement du document.
4. Le texte du paragraphe de la `div` d'identifiant `intro` est mis dans une balise `em`, à l'exception du texte dans la balise `b`, au chargement du document.
5. Si l'âge saisi est inférieur à 18, les deux questions concernant le permis de conduire seront masquées.
6. Si l'utilisateur à coché `Oui` à la question "`Avez-vous le permis de conduire ?`" alors la question d'après ("`Si non, pourquoi ?`") sera masquée.

Exercice 2. (Tableaux, Objets et DOM)
Le fichier **bdf.html** importe le fichier **bdf.js**. Compléter ce dernier sans modifier le premier pour obtenir le résultat illustré en Figure 1. Utiliser Firefox pour tester votre code avec la console.
Le fichier **bdf.js** définit :

- La variable `societes` qui est un tableau de tableaux, chacun stockant le nom d'une société de boules de fort, son numéro de téléphone, code postal, commune et président (ou présidente). Chaque président est identifié par son prénom suivi de son nom.
- La variable `prenoms_feminins` qui est un tableau de prénoms féminins.
- La variable `departements` qui est un tableau de numéros de départements français.
- La méthode de tableaux `supprimerDoublons()` qui renvoie une copie du tableau sur lequel elle est invoquée en en supprimant les doublons.

1. Calculer le nombre de sociétés dont le numéro de téléphone est la chaîne `xx-xx-xx-xx-xx`. Stocker le résultat dans la variable `sans_telephone`.
2. Cconstruire l'objet `distribution` qui associe à chaque numéro de département figurant dans le tableau `departements` le nombre de sociétés de ce département (les deux premiers chiffres du code postal d'une société correspondent à son numéro de département).
3. Extraire du tableau `societes` les prénoms des présidents (ou présidentes), en supprimer les doublons et les trier en ordre alphabétique. Stocker le résultat dans le tableau `prenoms`.
4. Le fichier HTML contient un tableau à 3 colonnes libellées "SOCIETE", "CP" et "PRESIDENT(E)". Le compléter en ajoutant une ligne par société avec les champs attendus. Si le président d'une société a l'un des prénoms féminins enregistrés dans le tableau `prenoms_feminins`, attribuer à la case HTML correspondante la classe `presidente` qui coloriera automatiquement le fond de la cellule en rose via les règles CSS.
5. Le code JS fourni pour cette question construit un objet avec le constructeur `Societe` pour chaque société et invoque sur cet objet la méthode `toHTML` en lui passant la data-list HTML d'identifiant `societes`. Une fois ce code exécuté, le champ texte de la page Web suggère différentes options au visiteur dès qu'il saisit des caractères (voir Figure 1).

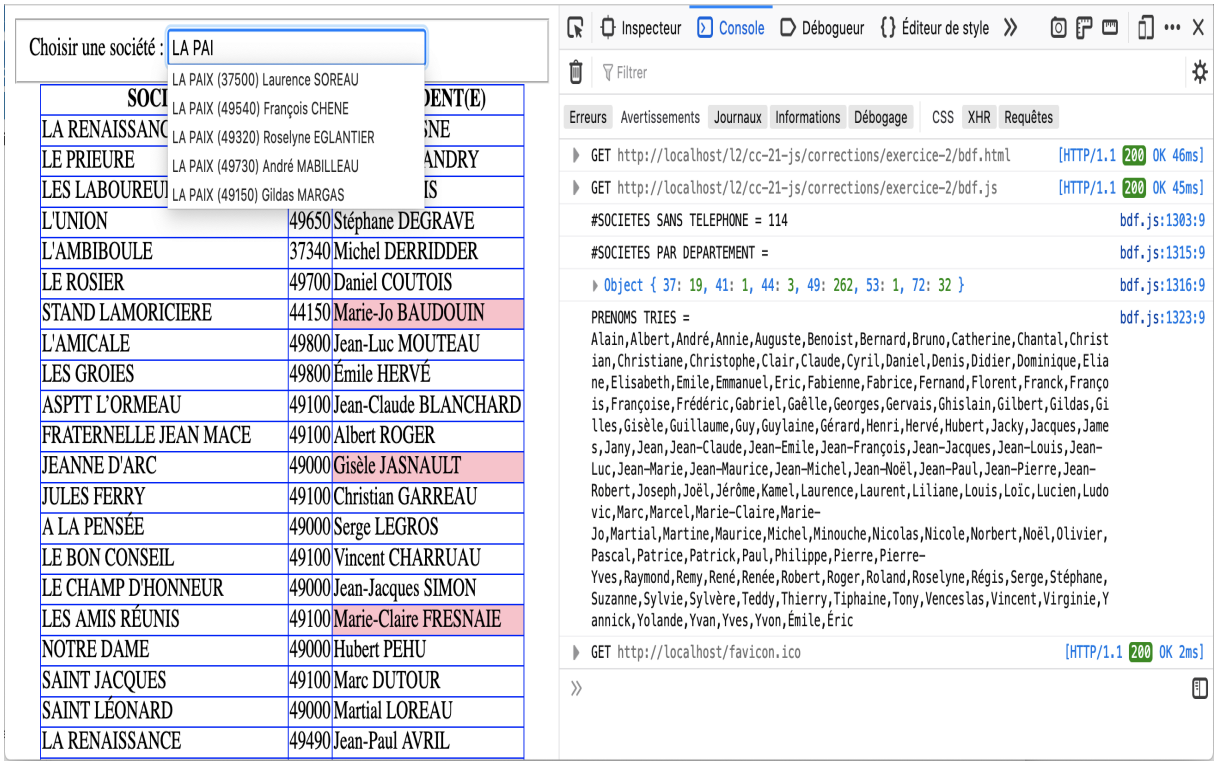

FIGURE 1 – Page web et affichage en console attendus

Implémenter le constructeur `Societe` qui stocke pour chaque objet créé le nom de la société, son code postal et son président. Ajouter ensuite, sans modifier le constructeur, la méthode `toHTML(datalist)` qui, lorsqu'elle est invoquée sur un objet `Societe` avec une data-list HTML en argument :

- Ajoute à cette data-list un sous-élément de balise `option`.
- Initialise l'attribut HTML `value` de cette option avec le texte concaténant le nom, code postal et nom de président stockés dans l'objet.